LAPORAN PRAKTIKUM 6
PEMROGRAMAN BERORIENTASI OBJEK

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Afdhal Dinilhak, M.Kom.

Asisten Praktikum :

Siti Fadhilah Rahmi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

A. Latihan

L1 Abstract Class dan Abstract Method

L1.a – Product

```java
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; // dalam rupiah

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name; }
    public long getPrice() { return price; }

    public long getFinalPrice() { return price; }

    public abstract String category();

    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" + getFinalPrice();
    }
}
```

Product dijadikan abstract class karena kasir tidak pernah menjual "produk" saja, melainkan produk fisik atau digital. Class ini menyimpan tiga field private final, yaitu sku, name, dan price (dalam rupiah, bertipe long). Constructor-nya protected karena hanya boleh dipanggil oleh subclass lewat super, dan di dalamnya ada validasi: SKU dan nama tidak boleh null atau kosong, harga tidak boleh negatif, dengan IllegalArgumentException bila dilanggar.

Ada tiga jenis anggota di dalam class ini. Pertama, getter getSku(), getName(), dan getPrice(). Kedua, getFinalPrice() yang mengembalikan harga dasar dan boleh di-override subclass dipakai DigitalProduct untuk diskon. Ketiga, abstract method category() yang tidak punya badan dan wajib diisi tiap subclass konkret. Method getInfo() adalah method biasa yang memanggil category() walaupun isinya belum

ada. Pemanggilan itu sah karena setiap objek nyata pasti berasal dari subclass konkret. Field dibuat private dengan getter agar subclass tidak bisa melewati validasi milik induk.

L1.b – PhysicalProduct

```java
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be positive");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

PhysicalProduct adalah subclass konkret dari Product untuk barang yang berwujud, misalnya kopi atau rendang kemasan. Constructor-nya menerima sku, name, price, dan weightGrams. Baris pertama constructor memanggil super(sku, name, price) karena bagian induk harus dibangun lebih dulu. Setelah itu berat divalidasi: weightGrams yang kurang dari atau sama dengan 0 ditolak dengan IllegalArgumentException.

Class ini meng-override category() dan mengembalikan "Fisik". Class ini juga mengimplementasikan interface Shippable sehingga memiliki getWeightGrams() dengan anotasi @Override. Berat disimpan sebagai field private final.

L1.c – DigitalProduct

```java
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;

    public DigitalProduct(String sku, String name, long price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) {
            throw new IllegalArgumentException("Download URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public String category() {
        return "Digital";
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

DigitalProduct adalah subclass konkret Product untuk barang tak berwujud seperti e-book. Constructor-nya menerima sku, name, price, dan downloadUrl. Setelah super dipanggil, downloadUrl divalidasi agar tidak null atau kosong. Class ini menyediakan getDownloadUrl() dan meng-override category() dengan nilai "Digital".

File di atas adalah versi akhir. Bagian implements Discountable, konstanta DIGITAL_DISCOUNT_PERCENT, discountPercent(), dan getFinalPrice(). DigitalProduct tidak mengimplementasikan Shippable karena produk digital tidak dikirim.

L1.d – ShippingService

```java
package shop.shipping;

public abstract class ShippingService {
    private final String name;

    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() {
        return name;
    }

    protected abstract long ratePerKg();

    public long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

ShippingService adalah abstract class untuk semua kurir. Semua kurir memakai rumus yang sama, sedangkan yang berbeda hanya tarif per kilogram. Karena itu tarif dijadikan abstract method protected ratePerKg() yang diisi subclass, dan rumus ditulis sekali di method konkret calculateCost().

Pada calculateCost(), berat 0 gram atau kurang menghasilkan ongkir 0. Selain itu, berat dibulatkan ke atas per kilogram dengan rumus (weightGrams + 999L) / 1000, sehingga 700 gram menjadi 1 kg dan 1001 gram menjadi 2 kg, lalu dikalikan tarif. Constructor protected menyimpan nama kurir setelah divalidasi tidak kosong, dan getName() menyediakan nama tersebut.

L1.e – RegularShipping dan ExpressShipping

```java
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super(name: "Reguler");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

RegularShipping adalah kurir reguler. Constructor-nya memanggil super("Reguler") dan ratePerKg() di-override dengan tarif 10_000 rupiah per kg. Anotasi @Override memastikan compiler memeriksa bahwa method ini memang menimpa method abstract milik induk. Class ini disediakan oleh modul.

```
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super(name: "Ekspres");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }
}
```

ExpressShipping ditulis sendiri dengan pola yang sama, yaitu nama "Ekspres" dan tarif 18_000 rupiah per kg. Rumus pembulatan tidak ditulis ulang karena diwarisi dari ShippingService. Dengan berat 700 gram, ongkir Reguler Rp10.000 dan Ekspres Rp18.000.

L2 Interface dan Program to an Interface

L2.a – Shippable

```
1   package shop.shipping;
2
3   public interface Shippable {
4       int getWeightGrams();
5   }
```

Shippable adalah interface kemampuan: apa pun yang punya berat dan bisa dikirim. Isinya hanya satu method, getWeightGrams(). Method interface otomatis public dan abstract, sehingga tidak perlu ditulis. PhysicalProduct diubah menjadi extends Product implements Shippable dan meng-override getWeightGrams() dengan @Override. DigitalProduct sengaja tidak mengimplementasikannya karena produk digital tidak dikirim.

L2.b – PaymentMethod

```java
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);

    default long adminFee(long amount) {
        return 0;
    }
}
```

PaymentMethod adalah kontrak untuk semua metode pembayaran. Setiap implementasi wajib punya getName() dan pay(long amount) yang mengembalikan true bila pembayaran berhasil. Kasir hanya butuh tombol "bayar sejumlah sekian" dan tidak perlu tahu cara kerja bank atau dompet digital.

L2.c – EWalletPayment

```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

EWalletPayment adalah pembayaran dengan saldo dompet digital. Constructor menerima provider dan balance, serta menolak provider kosong dan saldo negatif. getName() mengembalikan "E-Wallet " + provider, dan getBalance() menyediakan saldo saat ini.

Pada pay(amount), pembayaran gagal (false) bila nominal tidak positif atau melebihi saldo. Bila berhasil, saldo dikurangi lalu mengembalikan true. Method

refund() dan implements RefundablePayment ditambahkan pada L3.d, sehingga file di atas adalah versi akhirnya.

L2.d – BankTransferPayment

```java
package shop.payment;

public class BankTransferPayment implements PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
        if (bankName == null || bankName.isBlank()) {
            throw new IllegalArgumentException("Bank name must not be blank");
        }
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }
}
```

BankTransferPayment adalah simulasi pembayaran transfer bank. Constructor menerima bankName dan menolak nilai kosong. getName() mengembalikan "Transfer " + bankName. Method pay(amount) hanya mengembalikan amount > 0 karena ini simulasi tanpa saldo. Class ini sengaja hanya implements PaymentMethod, tanpa refund, sehingga transfer bank tidak bisa di-refund.

L2.e – CheckoutService

```java
public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        long sub = subtotal(items);
        int weight = totalWeight(items);
        long shippingCost = (shipping != null) ? shipping.calculateCost(weight) : 0;
        return sub + shippingCost;
    }

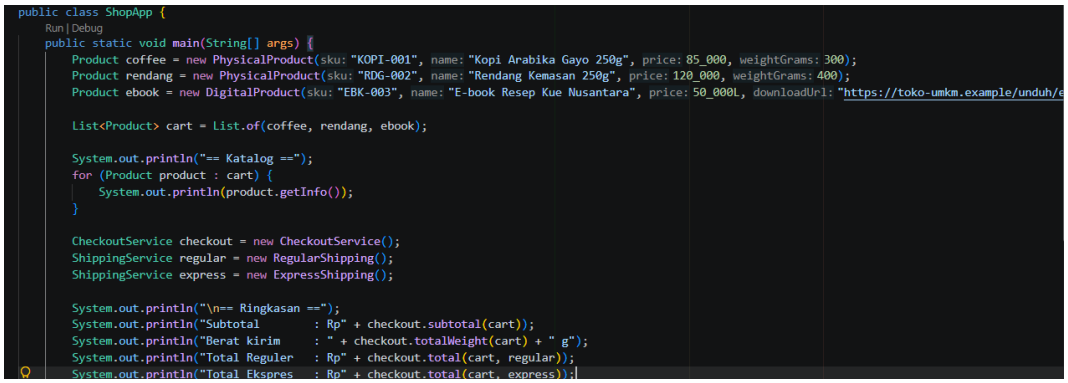
    public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        long finalTotal = total(items, shipping);
        return payment.pay(finalTotal);
    }

    public boolean refund(PaymentMethod payment, long amount) {
        if (!(payment instanceof RefundablePayment refundablePayment)) {
            return false;
        }
        return refundablePayment.refund(amount);
    }
}
```

CheckoutService menghitung total belanja dan memprosesnya. subtotal() dan totalWeight() disediakan modul. subtotal() menjumlahkan getFinalPrice() setiap produk. totalWeight() menjumlahkan berat hanya untuk produk yang bertipe Shippable; yang diperiksa adalah kemampuan, bukan class tertentu seperti PhysicalProduct.

Dua method sisanya ditulis sendiri. total() adalah subtotal ditambah ongkir dari ShippingService untuk total berat. checkout() mengembalikan false bila keranjang null atau kosong, selain itu membayar total() dengan payment.pay. Inilah program to an interface: class ini hanya mengenal ShippingService dan PaymentMethod, sehingga metode pembayaran atau kurir baru bisa ditambahkan tanpa mengubah CheckoutService.

L2.f – ShopApp



ShopApp adalah program utama yang mencoba semuanya. Tiga produk dibuat dengan tipe variabel Product (dua PhysicalProduct dan satu DigitalProduct) lalu dimasukkan ke List<Product>. Perulangan mencetak getInfo() tiap produk; polimorfisme membuat category() yang dipanggil adalah milik masing-masing subclass. Variabel regular dan express bertipe ShippingService sehingga CheckoutService tidak perlu tahu kurir konkretnya. Bagian pembayaran, struk, dan voucher ditambahkan pada L3.e dan L5.c.

L3 Default, Static, Private Method dan Interface Inheritance

L3.a – Discountable

```java
package shop.discount;

public interface Discountable {
    int discountPercent();

    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }
}
```

Discountable adalah aturan promo: harga dikurangi sekian persen, dan persentase tidak boleh negatif atau di atas 100. Interface ini memuat keempat jenis method. discountPercent() adalah abstract method sehingga setiap class menentukan persentasenya sendiri. applyDiscount(price) adalah default method yang menghitung harga setelah diskon. percentOf(amount, percent) adalah static method

utilitas. safePercent() adalah private method yang membatasi persentase ke rentang 0 sampai 100, dan tidak terlihat dari luar interface.

L3.b – DigitalProduct

```java
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;

    public DigitalProduct(String sku, String name, long price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) {
            throw new IllegalArgumentException("Download URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public String category() {
        return "Digital";
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

DigitalProduct diubah menjadi extends Product implements Discountable. discountPercent() mengembalikan konstanta private static final DIGITAL_DISCOUNT_PERCENT = 10. getFinalPrice() di-override agar mengembalikan applyDiscount(getPrice()), sehingga harga e-book Rp50.000 menjadi Rp45.000. CheckoutService tidak perlu diubah karena subtotal() sudah memanggil getFinalPrice() (polimorfisme). Akibatnya subtotal keranjang contoh menjadi Rp250.000 dan total Reguler Rp260.000.

L3.c.1 – RefundablePayment

```java
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);
}
```

RefundablePayment adalah interface turunan yang memperluas PaymentMethod dengan kata kunci extends (bukan implements). Interface ini menambah satu kemampuan, yaitu refund(amount) yang mengembalikan true bila pengembalian dana berhasil. Setiap class yang mengimplementasikannya wajib memenuhi seluruh kontrak PaymentMethod ditambah refund.

L3.c.2 – InstallmentPayment

```java
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException(
                "Months must be between 1 and " + maxInstallmentMonths()
            );
        }
        return amount / months;
    }
}
```

InstallmentPayment juga memperluas PaymentMethod dan menambah kemampuan cicilan. maxInstallmentMonths() adalah abstract method yang menentukan batas bulan. monthlyInstallment(amount, months) adalah default method yang menolak bulan di luar rentang 1 sampai maxInstallmentMonths() dengan IllegalArgumentException, lalu menghitung cicilan bulanan. Menurut modul cicilan dibulatkan ke atas dengan (amount + months - 1) / months (contoh 100000 dibagi 3 menjadi 33334). Pada kode proyek ini rumusnya amount / months tanpa pembulatan, sehingga hasilnya 33333 dan pemeriksaan L3.5 gagal.

L3.d.1 – EWalletPayment

```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

EWalletPayment diubah menjadi implements RefundablePayment. refund(amount) mengembalikan false bila nominal tidak positif, selain itu menambah saldo lalu mengembalikan true. Dengan begitu refund e-book Rp45.000 menaikkan saldo dari Rp40.000 menjadi Rp85.000.

L3.d.2 – CreditCardPayment

```java
package shop.payment;

public class CreditCardPayment implements RefundablePayment, InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        if (cardHolder == null || cardHolder.isBlank()) {
            throw new IllegalArgumentException("Card holder must not be blank");
        }
        if (creditLimit <= 0) {
            throw new IllegalArgumentException("Credit limit must be positive");
        }
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    public long getUsedCredit() {
        return usedCredit;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || (usedCredit + amount) > creditLimit) {
            return false;
        }
        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }
}
```

CreditCardPayment mengimplementasikan dua interface sekaligus, yaitu RefundablePayment dan InstallmentPayment. Java tidak mengizinkan extends lebih dari satu class, tetapi mengizinkan implements banyak interface. Constructor menerima cardHolder dan creditLimit serta menolak pemegang kartu kosong dan limit yang tidak positif. Field usedCredit mulai dari 0.

pay() gagal bila nominal tidak positif atau usedCredit + amount melebihi creditLimit; bila berhasil usedCredit bertambah. refund() gagal bila nominal tidak

positif atau melebihi usedCredit; bila berhasil usedCredit berkurang. maxInstallmentMonths() mengembalikan 12, dan getUsedCredit() menyediakan pemakaian kredit saat ini. getName() mengembalikan "Kartu Kredit (" + cardHolder + ")".

L3.d.3 – CheckoutService

```
public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        long sub = subtotal(items);
        int weight = totalWeight(items);
        long shippingCost = (shipping != null) ? shipping.calculateCost(weight) : 0;
        return sub + shippingCost;
    }

    public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        long finalTotal = total(items, shipping);
        return payment.pay(finalTotal);
    }

    public boolean refund(PaymentMethod payment, long amount) {
        if (!(payment instanceof RefundablePayment refundablePayment)) {
            return false;
        }
        return refundablePayment.refund(amount);
    }
}
```

Method refund ditambahkan ke CheckoutService untuk meneruskan pengembalian dana ke payment.refund(amount). Pada kode proyek, parameternya bertipe PaymentMethod dan tipenya diperiksa dengan instanceof di dalam method. Modul menghendaki parameter bertipe RefundablePayment sehingga compiler sendiri yang menolak refund untuk metode yang tidak mendukungnya. Karena perbedaan ini, pemeriksaan L3.6 gagal.

L3.e – ShopApp

```
EWalletPayment wallet = new EWalletPayment(provider: "DompetKu", balance: 300_000);
PaymentMethod transfer = new BankTransferPayment(bankName: "Bank Nagari");

System.out.println("\n== Pembayaran ==");
System.out.println(wallet.getName() + " berhasil? " + checkout.checkout(cart, regular, wallet));
System.out.println("Sisa saldo: Rp" + wallet.getBalance());
System.out.println(transfer.getName() + " berhasil? " + checkout.checkout(cart, express, transfer));

CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1_000_000);
System.out.println(card.getName() + " berhasil? " + checkout.checkout(cart, express, card));
System.out.println("Cicilan 3 bulan: Rp" + card.monthlyInstallment(checkout.total(cart, express), months: 3));

System.out.println("\n== Refund e-book ==");
System.out.println("Refund e-wallet berhasil? " + checkout.refund(wallet, ebook.getFinalPrice()));
System.out.println("Saldo setelah refund: Rp" + wallet.getBalance());
```

Bagian ini mencoba tiga metode pembayaran. E-wallet Rp300.000 membayar total Reguler Rp260.000 sehingga sisa Rp40.000. Transfer bank membayar total Ekspres Rp268.000 dan berhasil. Kartu kredit dengan limit Rp1.000.000 membayar Rp268.000 dan berhasil. Cicilan 3 bulan dari Rp268.000 adalah Rp89.333 pada kode ini (Rp89.334 menurut modul karena pembulatan ke atas). Terakhir, refund e-book Rp45.000 ke e-wallet menaikkan saldo menjadi Rp85.000.

L4 Final Method dan Final Class

L4.a –ShippingService

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;

    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() {
        return name;
    }

    protected abstract long ratePerKg();

    public long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

meminta calculateCost() dijadikan final agar kurir hanya boleh mengatur tarif lewat ratePerKg() dan tidak bisa mengganti rumus, misalnya tidak membulatkan berat. Pada kode proyek, method ini masih tanpa final. Untuk menguji, dibuat sementara class FreeShipping yang meng-override calculateCost(); class tersebut berhasil dikompilasi,

L4.b – Address

```java
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {
        if (street == null || street.isBlank()) {
            throw new IllegalArgumentException("Street must not be blank");
        }
        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City must not be blank");
        }
        if (postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Postal code must not be blank");
        }
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() {
        return street;
    }

    public String getCity() {
        return city;
    }

    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Address menyimpan alamat pengiriman. Class ini final dengan field private final street, city, dan postalCode, serta tanpa setter, sehingga nilainya tidak pernah berubah setelah dibuat (immutable). Class-nya juga dibuat final agar tidak ada subclass seperti EditableAddress yang menambahkan setter dan membocorkan jaminan immutable. Constructor menolak nilai null atau kosong, dan toString() menghasilkan street + ", " + city + " " + postalCode.

L5 Nested Class

L5.a – Order

```java
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank() || shippingAddress == null) {
            throw new IllegalArgumentException("Order id and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() { return orderId; }
    public Address getShippingAddress() { return shippingAddress; }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() { return Collections.unmodifiableList(lines); }
```

Order mewakili satu pesanan. Constructor menolak orderId kosong dan alamat null. addLine() membuat objek Line baru dan memasukkannya ke daftar. getLines() mengembalikan Collections.unmodifiableList agar daftar asli tidak bisa diubah dari luar. bagian yang ditulis sendiri adalah Line, getItemsTotal(), Receipt, dan createReceipt() di bawah.

L5.b.1 – Line

```java
public static class Line {
    private final Product product;
    private final int quantity;

    public Line(Product product, int quantity) {
        if (product == null || quantity <= 0) {
            throw new IllegalArgumentException("Product must not be null and quantity must be positive");
        }
        this.product = product;
        this.quantity = quantity;
    }

    public Product getProduct() { return product; }
    public int getQuantity() { return quantity; }

    public long subtotal() {
        return product.getFinalPrice() * quantity;
    }
}
```

Line adalah satu baris pesanan, misalnya "Kopi Gayo x2". Line dibuat static nested class karena hanya butuh produk dan jumlah, dan tidak membaca data Order. Dengan static, Line tidak menyimpan referensi tersembunyi ke Order dan bisa

dibuat tanpa objek Order (new Order.Line(coffee, 1)). Constructor menolak produk null dan jumlah tidak positif. subtotal() adalah harga akhir produk dikali jumlah.

L5.b.2 – getItemsTotal()

```
public long getItemsTotal() {
    long total = 0;
    for (Line line : lines) {
        total += line.subtotal();
    }
    return total;
}
```

getItemsTotal() menjumlahkan subtotal() semua baris pesanan. Untuk pesanan contoh (kopi x2 Rp85.000 dan e-book x1 Rp45.000) hasilnya Rp215.000.

L5.b.3 – Receipt

```
public class Receipt {
    public String toText() {
        StringBuilder sb = new StringBuilder();
        sb.append("STRUK PESANAN ").append(Order.this.orderId).append("\n");
        for (Line line : lines) {
            sb.append("- ").append(line.getProduct().getName())
              .append(" x").append(line.getQuantity())
              .append(" = Rp").append(line.subtotal()).append("\n");
        }
        sb.append("Total pesanan: Rp").append(getItemsTotal()).append("\n");
        sb.append("Kirim ke: ").append(Order.this.shippingAddress);
        return sb.toString();
    }
}
```

Receipt adalah struk yang selalu milik satu pesanan tertentu, sehingga dibuat inner class (tanpa static). Class ini membaca orderId dan shippingAddress milik objek luarnya lewat Order.this. Di dalam inner class, this adalah objek Receipt, bukan Order. toText() membangun teks struk: judul dengan id pesanan, satu baris per item, total pesanan, dan alamat tujuan. Objek Receipt hanya bisa dibuat dari objek Order (order.new Receipt()).

L5.b.4 – createReceipt()

```
public Receipt createReceipt() {
    return new Receipt();
}
```

createReceipt() adalah cara yang rapi membuat struk dari dalam Order, karena di dalam class luar objek inner bisa dibuat dengan new Receipt() biasa tanpa sintaks order.new.

L5.c – ShopApp

```
Address address = new Address(street: "Jl. Moh Hatta No. 10", city: "Padang", postalCode: "25121");
Order order = new Order(orderId: "ORD-0001", address);
order.addLine(coffee, quantity: 2);
order.addLine(ebook, quantity: 1);

Order.Receipt receipt = order.new Receipt();
System.out.println("\n" + receipt.toText());

PaymentMethod voucher = new VoucherPayment(remaining: 50_000);

System.out.println("\n" + voucher.getName() + " untuk e-book berhasil? "
    + checkout.checkout(List.of(ebook), regular, voucher));


vate static class VoucherPayment implements PaymentMethod {
 private long remaining;

 private VoucherPayment(long remaining) {
     this.remaining = remaining;
 }

 @Override
 public String getName() {
     return "Voucher Promo Hari Ini";
 }

 @Override
 public boolean pay(long amount) {
     if (amount <= 0 || amount > remaining) {
         return false;
     }
     remaining -= amount;
     return true;
 }
```

Bagian ini membuat Address dan Order, menambahkan dua baris, lalu membuat struk dengan order.new Receipt(), sintaks yang menunjukkan bahwa objek inner hanya bisa lahir dari objek luar. Voucher promo Rp50.000 adalah implementasi PaymentMethod yang hanya dipakai sekali. Pada kode proyek, voucher ditulis sebagai private static nested class bernama VoucherPayment. Modul meminta bentuk anonymous class, sehingga pemeriksaan L5.3 gagal. Perilaku programnya sama: voucher berhasil membayar e-book Rp45.000 tanpa ongkir karena produk digital tidak dikirim.

Output dan Hasil Selfcheck

Output :

```
PS D:\Ditzz\VisualStudio\Praktikum6\latihan\abstraction-2511533002> java -cp bin shop.app.ShopApp
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal        : Rp250000
Berat kirim     : 700 g
Total Reguler   : Rp260000
Total Ekspres   : Rp268000

== Pembayaran ==
E-Wallet DompetKu berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89333

== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000

STRUK PESANAN ORD-0001
- Kopi Arabika Gayo 250g x2 = Rp170000
- E-book Resep Kue Nusantara x1 = Rp45000
Total pesanan: Rp215000
Kirim ke: Jl. Moh Hatta No. 10, Padang 25121

Voucher Promo Hari Ini untuk e-book berhasil? true
```

Selfcheck :

```
PS D:\Ditzz\VisualStudio\Praktikum6\latihan\abstraction-2511533002> java -cp bin selfcheck.SelfCheck
=== SELF CHECK: Praktikum Abstraction ===


-- L1 - Abstract class & abstract method
  [LULUS] L1.1 Product adalah abstract class
  [LULUS] L1.2 Product punya abstract method category()
  [LULUS] L1.3 PhysicalProduct & DigitalProduct adalah subclass konkret Product
  [LULUS] L1.4 Konstruktor menolak harga negatif (IllegalArgumentException)
  [LULUS] L1.5 getInfo() PhysicalProduct sesuai format
  [LULUS] L1.6 ShippingService abstract dengan abstract method ratePerKg()
  [LULUS] L1.7 calculateCost(): Reguler 700g=10000, 1001g=20000, 0g=0; Ekspres 700g=18000

-- L2 - Interface & program to an interface
  [LULUS] L2.1 Shippable interface; PhysicalProduct implements, DigitalProduct tidak
  [LULUS] L2.2 PaymentMethod interface dengan getName() dan pay(long)
  [LULUS] L2.3 EWalletPayment: bayar sukses mengurangi saldo, saldo kurang ditolak
  [LULUS] L2.4 CheckoutService.checkout(...) menerima PaymentMethod & ShippingService (bukan class konkret)
  [LULUS] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000

-- L3 - Default, static, private method; interface inheritance
  [LULUS] L3.1 Discountable: applyDiscount default, percentOf static, ada private method
  [LULUS] L3.2 DigitalProduct implements Discountable; harga akhir 50000 -> 45000
  [LULUS] L3.3 RefundablePayment & InstallmentPayment adalah interface turunan PaymentMethod
  [LULUS] L3.4 CreditCardPayment implements RefundablePayment DAN InstallmentPayment
  [GAGAL] L3.5 monthlyInstallment(100000, 3)=33334 dan 13 bulan ditolak
          -> hasil tidak sesuai spesifikasi
  [GAGAL] L3.6 CheckoutService.refund(...) hanya menerima RefundablePayment
          -> method/konstruktor tidak ditemukan: shop.service.CheckoutService.refund(shop.payment.RefundablePayment,long
)

-- L4 - final method & final class
  [GAGAL] L4.1 ShippingService.calculateCost(int) adalah final method
          -> hasil tidak sesuai spesifikasi
  [LULUS] L4.2 Address final class, semua field private final, tanpa setter

-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 20 / 23 pemeriksaan lulus ===
Belum lulus:
  - L3.5 monthlyInstallment(100000, 3)=33334 dan 13 bulan ditolak (hasil tidak sesuai spesifikasi)
  - L3.6 CheckoutService.refund(...) hanya menerima RefundablePayment (method/konstruktor tidak ditemukan: shop.service.
CheckoutService.refund(shop.payment.RefundablePayment,long))
  - L4.1 ShippingService.calculateCost(int) adalah final method (hasil tidak sesuai spesifikasi)
```

B. Tugas

E.0 Parameter pribadi berbasis NIM

```
PS D:\Ditzz\VisualStudio\Praktikum6\latihan\abstraction-2511533002> java -cp bin tools.NimParams 2511533002
Parameter tugas untuk NIM 2511533002
  CARD_ADMIN_PERCENT  = 3
  TRANSFER_ADMIN_FEE  = 3500
  SMS_MAX_LENGTH      = 102
```

T1.a – Cara yang salah dulu

```
ShopApp.java Latihan\abstraction-2511533002\src\shop\app 1
  The type ShopApp.VoucherPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 71, Col 26]
BankTransferPayment.java Latihan\abstraction-2511533002\src\shop\payment 1
  The type BankTransferPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 3, Col 14]
CreditCardPayment.java Latihan\abstraction-2511533002\src\shop\payment 1
  The type CreditCardPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 3, Col 14]
EWalletPayment.java Latihan\abstraction-2511533002\src\shop\payment 1
  The type EWalletPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 3, Col 14]
PaymentMethod.java Latihan\abstraction-2511533002\src\shop\payment 1
  Abstract methods do not specify a body Java(603979889) [Ln 10, Col 10]
```

Tambahkan method abstract long adminFee(long amount); ke PaymentMethod. Terjadi 5 error pada class ShopApp, BankTransferPayment, CreditCardPayment, EWalletPayment. Dan PaymentMethod

T1.b - Default method

```java
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);

    default long adminFee(long amount) {
        return 0;
    }
}
```

Dengan mengubah method menjadi default, semua kendala kompilasi akan terselesaikan secara otomatis tanpa menyentuh kelas-kelas yang mengimplementasikannya.

T1.c - Override sesuai kebutuhan.

T1.c.1 - CreditCardPayment

```java
@Override
public long adminFee(long amount) {
return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
```

Method ini digunakan agar kartu kredit membebankan biaya admin sebesar persentase dari jumlah transaksi, yaitu amount * CARD_ADMIN_PERCENT / 100. Dengan persentase 3%, transaksi Rp200.000 dikenai biaya Rp6.000.

T1.c.2 - BankTransferPayment

```
@Override
public long adminFee(long amount) {
return StudentInfo.TRANSFER_ADMIN_FEE;
```

Method ini digunakan agar transfer bank membebankan biaya admin tetap sebesar TRANSFER_ADMIN_FEE (Rp3.500), berapa pun jumlah transaksinya.

T1.d - CheckoutService

```
public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    if (items == null || items.isEmpty()) {
    return false;
    }
    long finalTotal = grandTotal(items, shipping, payment);
    return payment.pay(finalTotal);
}

public boolean refund(PaymentMethod payment, long amount) {
    if (!(payment instanceof RefundablePayment refundablePayment)) {
        return false;
    }
    return refundablePayment.refund(amount);
}

public long grandTotal(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    long baseTotal = total(items, shipping);
    return baseTotal + payment.adminFee(baseTotal);
}
```

Menambahkan public long grandTotal Method ini digunakan agar checkout membayar grandTotal.

T1.e – Konflik diamond

T1.e.1 - RefundablePayment

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);

    default String policy() {
        return "Refund maksimal 7 hari";
    }
}
```

Tambahkan default method policy():

T1.e.2 - InstallmentPayment

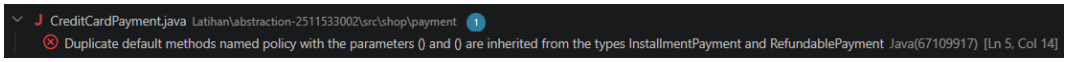
```java
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException(
                "Months must be between 1 and " + maxInstallmentMonths()
            );
        }
        return amount / months;
    }

    default String policy() {
        return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
    }
}
```

Tambahkan default method policy() dengan nama yang sama



Setelah dua default method di atas ditambahkan, kompilasi menghasilkan error

T1.e.3 - CreditCradPayment

```java
@Override
public String policy() {
return RefundablePayment.super.policy() + "; " + InstallmentPayment.super.policy();
}
```

override policy() di CreditCardPayment

Pertanyaan T1.e

1) Mengapa EWalletPayment tidak perlu meng-override policy()?

   Karena EWalletPayment hanya mengimplementasikan RefundablePayment—di mana method policy() pada *interface* tersebut sudah berstatus *default*—maka kelas yang mengimplementasikannya tidak diwajibkan untuk melakukan *override*.

2) Mengapa Java tidak memilih salah satu default secara otomatis?

   Untuk menghindari ambiguitas serta cacat logika, perlu dicatat bahwa EWalletPayment hanya mengimplementasikan RefundablePayment.

Mengingat method policy() sudah bersifat *default*, kelas terkait tidak harus melakukan *override*.

Checkpoint T1

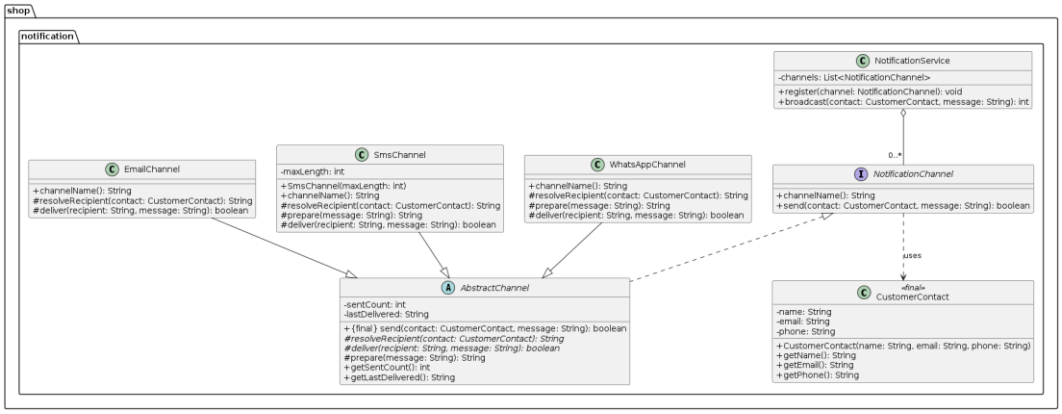
```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511533002 | Rifqi Aditya

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (3%, Rp3500, 102 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
```

T2 – Merancang sistem notifikasi pesanan

Langkah 1 – Rancang dulu (UML)



Langkah 2 – Implementasikan

CustomerContact

```java
package shop.notification;

public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;

    public CustomerContact(String name, String email, String phone) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
        this.email = (email == null) ? "" : email;
        this.phone = (phone == null) ? "" : phone;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPhone() { return phone; }
}
```

Constructor CustomerContact(String name, String email, String phone), tujuan = menyimpan data kontak pelanggan. name wajib diisi; jika null atau kosong, constructor melempar IllegalArgumentException. email dan phone boleh kosong, dan nilai null diubah menjadi “”. Class ini final dan hanya punya getter (getName, getEmail, getPhone), sehingga datanya tidak bisa diubah setelah dibuat.

NotificationChannel

```java
package shop.notification;

public interface NotificationChannel {
    String channelName();

    boolean send(CustomerContact contact, String message);
}
```

Interface NotificationChannel, tujuan = kontrak yang harus dipenuhi semua kanal notifikasi. channelName() mengembalikan nama kanal, dan send(CustomerContact contact, String message) mengembalikan true jika pesan berhasil dikirim.

AbstractChannel

```java
package shop.notification;

public abstract class AbstractChannel implements NotificationChannel {
    private int sentCount;
    private String lastDelivered;

    @Override
    public final boolean send(CustomerContact contact, String message) {
        if (contact == null || message == null || message.isBlank()) {
            return false;
        }
        String recipient = resolveRecipient(contact);
        if (recipient == null || recipient.isEmpty()) {
            return false;
        }
        String prepared = prepare(message);
        if (!deliver(recipient, prepared)) {
            return false;
        }
        sentCount++;
        lastDelivered = prepared;
        return true;
    }

    /** @return alamat tujuan untuk kanal ini, atau null/"" jika kontak tidak cocok */
    protected abstract String resolveRecipient(CustomerContact contact);

    /** Mengirim pesan yang sudah disiapkan (simulasi). */
    protected abstract boolean deliver(String recipient, String message);

    /** Hook: subclass boleh meng-override untuk mengubah pesan. */
    protected String prepare(String message) {
        return message;
    }

    public int getSentCount() {
        return sentCount;
    }

    public String getLastDelivered() {
        return lastDelivered;
    }
}
```

Class abstract AbstractChannel implements NotificationChannel, tujuan = menyediakan alur pengiriman yang sama untuk semua kanal. Method send(...) bersifat final, sehingga urutannya tidak bisa diubah subclass: menolak pesan kosong, mencari alamat tujuan (resolveRecipient), menyiapkan pesan (prepare), mengirim (deliver), lalu mencatat. Jika kontak atau pesan tidak valid, atau alamat tujuan tidak ditemukan, send mengembalikan false. Setelah berhasil dikirim, sentCount bertambah satu dan pesan terakhir disimpan, yang bisa dibaca lewat getSentCount() dan getLastDelivered(). resolveRecipient dan deliver bersifat protected abstract dan diisi tiap kanal, sedangkan prepare adalah method protected biasa yang boleh di-override. Pola ini adalah Template Method.

EmailChannel

```java
package shop.notification;

public class EmailChannel extends AbstractChannel {

    @Override
    public String channelName() {
        return "Email";
    }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String email = contact.getEmail();
        return email.contains("@") ? email : null;
    }

    @Override
    protected boolean deliver(String recipient, String message) {
        return true; // simulasi: anggap email terkirim
    }
}
```

Constructor EmailChannel tanpa parameter, tujuan = email kontak, hanya jika memuat @. Jika email tidak memuat @, pengiriman ditolak dan send mengembalikan false. channelName mengembalikan “Email”.

SmsChannel

```java
package shop.notification;

public class SmsChannel extends AbstractChannel {
    private static final String ELLIPSIS = "...";
    private final int maxLength;

    public SmsChannel(int maxLength) {
        if (maxLength < ELLIPSIS.length() + 1) {
            throw new IllegalArgumentException("maxLength is too small");
        }
        this.maxLength = maxLength;
    }

    @Override
    public String channelName() {
        return "SMS";
    }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        return phone.startsWith("08") ? phone : null;
    }

    @Override
    protected String prepare(String message) {
        if (message.length() <= maxLength) {
            return message;
        }
        return message.substring(0, maxLength - ELLIPSIS.length()) + ELLIPSIS;
    }

    @Override
    protected boolean deliver(String recipient, String message) {
        return true; // simulasi: anggap SMS terkirim
    }
}
```

Constructor SmsChannel(int maxLength), tujuan = telepon dengan awalan 08, pesan yang lebih panjang dari maxLength akan terpotong menjadi tepat maxLength karakter dan diakhiri "...". Pemotongan dilakukan dengan mengambil maxLength - 3 karakter pertama lalu menambahkan "...", sehingga panjang akhirnya tepat maxLength (pada tugas ini 102 karakter). Pesan yang tidak lebih panjang dari maxLength dikirim apa adanya. channelName mengembalikan "SMS".

WhatsAppChannel

```java
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {
    private static final String PREFIX = "[Toko UMKM] ";

    @Override
    public String channelName() {
        return "WhatsApp";
    }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        boolean validPrefix = phone.startsWith("08") || phone.startsWith("+62");
        return (validPrefix && phone.length() >= 10) ? phone : null;
    }

    @Override
    protected String prepare(String message) {
        return PREFIX + message;
    }

    @Override
    protected boolean deliver(String recipient, String message) {
        return true; // simulasi: anggap pesan WhatsApp terkirim
    }
}
```

Constructor WhatsAppChannel tanpa parameter, tujuan = telepon berawalan 08 atau +62 dengan panjang minimal 10. Setiap pesan diberi awalan “[Toko UMKM] “ lewat method prepare. Kontak yang nomornya tidak memenuhi syarat ditolak. channelName mengembalikan “WhatsApp”.

NotificationService

```java
package shop.notification;

import java.util.ArrayList;
import java.util.List;

public class NotificationService {
    private final List<NotificationChannel> channels = new ArrayList<>();

    public void register(NotificationChannel channel) {
        if (channel == null) {
            throw new IllegalArgumentException("Channel must not be null");
        }
        channels.add(channel);
    }

    /** @return jumlah kanal yang berhasil mengirim */
    public int broadcast(CustomerContact contact, String message) {
        int delivered = 0;
        for (NotificationChannel channel : channels) {
            if (channel.send(contact, message)) {
                delivered++;
            }
        }
        return delivered;
    }
}
```

Class NotificationService, tujuan = mengelola banyak kanal sekaligus. register(NotificationChannel channel) menambahkan kanal ke daftar, dan melempar IllegalArgumentException jika kanalnya null. broadcast(CustomerContact contact, String message) mengirim pesan lewat semua kanal terdaftar dan mengembalikan jumlah kanal yang berhasil. Contoh: kontak

tanpa email dengan telepon +6281298765432 hanya diterima WhatsApp, sehingga broadcast mengembalikan 1.

Output

```
Tugas oleh Rifqi Aditya (2511533002)

== T1: Total dengan biaya admin (Ekspres) ==
E-Wallet DompetKu : Rp268000 (admin Rp0)
Transfer Bank Nagari : Rp271500 (admin Rp3500)
Kartu Kredit (Siti Rahma) : Rp276040 (admin Rp8040)

Kebijakan kartu kredit : Refund maksimal 7 hari; Cicilan maksimal 12 bulan
Kebijakan e-wallet     : Refund maksimal 7 hari

== T2: Notifikasi status pesanan ==
[EMAIL to siti@example.com] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih tel
ah berbelanja produk UMKM Sumatera Barat di toko kami! Pantau terus status pesanan Anda di aplikasi toko.
[SMS to 081234567890] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 ...
[WA to 081234567890] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasi
h telah berbelanja produk UMKM Sumatera Barat di toko kami! Pantau terus status pesanan Anda di aplikasi toko.
[LOG] Siti Rahma: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbela
nja produk UMKM Sumatera Barat di toko kami! Pantau terus status pesanan Anda di aplikasi toko.
Terkirim ke Siti Rahma lewat 4 kanal
[WA to +6281298765432] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima ka
sih telah berbelanja produk UMKM Sumatera Barat di toko kami! Pantau terus status pesanan Anda di aplikasi toko.
[LOG] Budi: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja pr
oduk UMKM Sumatera Barat di toko kami! Pantau terus status pesanan Anda di aplikasi toko.
Terkirim ke Budi lewat 2 kanal
```

TugasCheck

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511533002 | Rifqi Aditya

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (3%, Rp3500, 102 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan

-- T2 - Sistem notifikasi
  [LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
  [LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx
```

Kemampuan *subclass* untuk meng-*override* semua method dapat memicu berbagai masalah, seperti kegagalan validasi, hilangnya rekam jejak , inkonsistensi alur program, hingga dilanggarnya batasan-batasan dari *parent class*.

3) Mengapa NotificationService menyimpan List, bukan List?

   Karena menerapkan konsep polymorphism

T3 – Code review BadShop

Output awal

```
PS D:\Ditzz\VisualStudio\Praktikum6\latihan\abstraction-2511533002> java -cp bin review.BadShop
Total cart: 170000
Ongkir EXPRES 2 kg: 0
ORD-1 -> Jl. Sudirman 10, Bukittinggi
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | Voucher, remaining balance 70000.0
Exception in thread "main" java.lang.UnsupportedOperationException: Voucher is not shipped
        at review.GiftVoucher.ship(BadShop.java:41)
        at review.BadShop.main(BadShop.java:184)
```

Masalahnya yaitu ongkir Ekspres 0, alamat ORD-1 ikut berubah, dan program berhenti dengan exception. Pada versi refactor semuanya hilang,

Masalah yang di temukan

No	Lokasi (Class/Method)	Masalah	Dampak nyata (bug atau risiko)	Perbaikan yang diusulkan
1	Courier.cost(String type, int kg) dan JneCourier	Abstract class tanpa method abstract. cost memakai switch pada String bebas, dan JneCourier hanya subclass kosong.	Salah ketik "EXPRES" jatuh ke default dan mengembalikan 0, sehingga ongkir gratis tanpa error. Menambah kurir baru berarti mengedit switch (melanggar OCP).	Jadikan ratePerKg() method protected abstract, dan cost(int kg) method final (template method). Buat RegularCourier, ExpressCourier, dan SameDayCourier.
2	Address (setter setStreet/setCity) dan	Address mutable dan dipakai bersama oleh dua pesanan.	Memanggil second.getAddress(). setCity("Bukittinggi") ikut mengubah	Jadikan Address immutable: field final, tanpa setter, dan method

	SimpleOrder		alamat ORD-1, karena keduanya menunjuk objek yang sama. Data pesanan lama rusak diam-diam.	withCity(...) yang membuat objek baru.
3	Discount.apply()	Default method memakai instanceof untuk mengecek subclass. Interface bergantung pada implementasinya sendiri.	Diskon baru harus mengedit interface (melanggar OCP). Tipe lain yang tidak terdaftar diam-diam tanpa diskon.	Jadikan ratePerKg() method protected abstract, dan cost(int kg) method final (template method). Buat RegularCourier, ExpressCourier, dan SameDayCourier.
4	Cart.Item	Nested class non-static menyimpan referensi tersembunyi ke objek Cart, padahal tidak memakainya.	Pemborosan memori dan coupling yang tidak perlu. Item tidak bisa dibuat tanpa instance Cart.	Tambah method abstract percent() di interface. apply() tetap sebagai default method tanpa instanceof: price - price * percent() / 100. Tiap diskon menentukan persentasenya sendiri.
5	GiftVoucher.balance bertipe double	Uang disimpan sebagai bilangan pecahan, sementara pay(long) memakai long.	Risiko kesalahan pembulatan pada nominal uang. Outputnya pun janggal: 70000.0.	Jadikan static final class Item, lalu buat dengan new Cart.Item(...).

Refactor BadShop

```java
package review.fixed;

import java.util.ArrayList;
import java.util.List;

interface Payment {
    boolean pay(long amount);

    String printInvoice();
}

interface Refundable extends Payment {
    boolean refund(long amount);
}

final class GiftVoucher implements Payment {
    private long balance = 100_000;

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public String printInvoice() {
        return "sisa Rp" + balance;
    }
}

abstract class Courier {
    protected abstract long ratePerKg();
    public final long cost(int kilograms) {
        return kilograms <= 0 ? 0 : kilograms * ratePerKg();
    }
}

final class RegularCourier extends Courier {
    @Override
    protected long ratePerKg() {
        return 10_000;
```

```
final class ExpressCourier extends Courier {
    @Override
    protected long ratePerKg() {
        return 18_000;
    }
}

final class SameDayCourier extends Courier {
    @Override
    protected long ratePerKg() {
        return 30_000;
    }
}

final class Cart {
    private final List<Item> items = new ArrayList<>();
    static final class Item {
        private final String name;
        private final long price;
        private final int quantity;

        Item(String name, long price, int quantity) {
            if (name == null || name.isBlank() || price < 0 || quantity <= 0) {
                throw new IllegalArgumentException("Item tidak valid");
            }
            this.name = name;
            this.price = price;
            this.quantity = quantity;
        }

        long subtotal() {
            return price * quantity;
        }
```

```
        String getName() {
            return name;
        }
    }

    void add(Item item) {
        items.add(item);
    }

    long total() {
        long total = 0;
        for (Item item : items) {
            total += item.subtotal();
        }
        return total;
    }
}
final class Address {
    private final String street;
    private final String city;

    Address(String street, String city) {
        this.street = street;
        this.city = city;
    }

    Address withCity(String newCity) {
        return new Address(street, newCity);
    }

    @Override
    public String toString() {
        return street + ", " + city;
    }
}

final class SimpleOrder {
    private final String id;
    private final Address address;
```

```java
final class SimpleOrder {
    SimpleOrder(String id, Address address) {
        this.id = id;
        this.address = address;
    }

    Address getAddress() {
        return address;
    }

    @Override
    public String toString() {
        return id + " -> " + address;
    }
}
interface Discount {
    int percent();

    default long apply(long price) {
        return price - price * percent() / 100;
    }
}

final class MemberDiscount implements Discount {
    @Override
    public int percent() {
        return 5;
    }
}

final class FlashSaleDiscount implements Discount {
    @Override
    public int percent() {
        return 30;
    }
}

public class BadShopRefactored {
    Run | Debug
    public static void main(String[] args) {
        Cart cart = new Cart();
        cart.add(new Cart.Item(name: "Kopi Gayo", price: 85_000, quantity: 2));
        System.out.println("Total cart: " + cart.total());

        Courier courier = new ExpressCourier();
```

```java
        System.out.println("Ongkir Ekspres 2 kg: " + courier.cost(kilograms: 2));

        Address home = new Address(street: "Jl. Sudirman 10", city: "Padang");
        SimpleOrder first = new SimpleOrder(id: "ORD-1", home);
        SimpleOrder second = new SimpleOrder(id: "ORD-2", home.withCity(newCity: "Bukittinggi"));
        System.out.println(first);
        System.out.println(second);

        Discount flashSale = new FlashSaleDiscount();
        System.out.println("Harga flash sale: " + flashSale.apply(price: 100_000));

        Payment voucher = new GiftVoucher();
        System.out.println(voucher.pay(amount: 30_000) + " | " + voucher.printInvoice());
    }
}
```

Output setelah refactor

```
Total cart: 170000
Ongkir Ekspres 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | sisa Rp70000
```

T4 – Refleksi: kapan memakai apa

Tabel 1 – Abstract class vs interface

Pertanyaan desain	Abstract class atau interface?	Alasan	Contoh dari proyek
Subclass berbagi field dan konstruktor	Abstract class	Abstract class boleh punya field dan konstruktor yang diwarisi subclass, sedangkan interface tidak bisa menyimpan data milik objek maupun punya konstruktor. Dengan begitu kode validasi dan penyimpanan data cukup ditulis sekali	Product menyimpan sku, name, dan price beserta konstruktor yang memvalidasinya, lalu diwarisi PhysicalProduct dan DigitalProduct. ShippingService menyimpan name dan diwarisi ExpressShipping.
Satu class perlu beberapa kemampuan sekaligus	Interface	Sebuah class hanya boleh extends satu class, tetapi boleh implements banyak interface. Karena itu beberapa kemampuan yang berbeda bisa digabung dalam satu class.	CreditCardPayment implements RefundablePayment, InstallmentPayment, sehingga kartu kredit punya kemampuan refund sekaligus cicilan.
Rumus bersama yang tidak boleh diubah subclass	Abstract class	Method di abstract class bisa dikunci dengan final, sehingga urutan langkahnya tidak bisa diubah subclass. Method default di interface masih bisa di-override, jadi tidak cocok untuk keperluan ini.	AbstractChannel.send(...) bersifat final dan menjalankan urutan tetap: validasi pesan, resolveRecipient, prepare, deliver, lalu pencatatan. EmailChannel, SmsChannel, dan WhatsAppChannel hanya mengisi langkah-langkahnya.

Kontrak yang perlu ditambah method baru di kemudian hari	Interface (dengan default method)	Method baru yang ditambahkan sebagai default langsung punya isi, sehingga class lama yang sudah mengimplementasikan interface tidak error dan tidak perlu diubah.	PaymentMethod.adminFee(long) ditambahkan belakangan sebagai default method yang mengembalikan 0. EWalletPayment tidak perlu diubah sama sekali dan otomatis tanpa biaya admin, sedangkan kartu kredit dan transfer bank meng-override-nya.

Tabel 2 – Jenis nested class

Kondisi	**Static nested, inner, atau anonymous?**	**Alasan**	**Contoh dari proyek**
Class pembantu tidak membaca data objek luar	Static nested	Static nested class tidak menyimpan referensi tersembunyi ke objek luar, sehingga lebih hemat dan bisa dibuat tanpa membuat objek luarnya terlebih dahulu.	Order.Line hanya memakai product dan quantity miliknya sendiri, tidak memakai field Order, dan dibuat dengan new Order.Line
Objek harus membaca field objek luarnya	Inner	Inner class otomatis memegang referensi ke objek luar (Order.this), sehingga bisa langsung membaca field dan method milik objek luar tanpa perlu menerimanya lewat parameter.	Order.Receipt.toText() membaca orderId, shippingAddress, lines, dan getItemsTotal() milik Order untuk mencetak struk. Receipt dibuat lewat order.createReceipt().

Implementasi sekali pakai, tidak perlu nama	Anonymous class	Class ini hanya dipakai di satu tempat, jadi tidak perlu diberi nama atau file sendiri. Interface langsung diimplementasikan di tempat objeknya dibuat.	Kanal log konsol di TugasApp: service.register(new NotificationChannel() , yang mencetak [LOG] nama: pesan, dipakai satu kali saja.

C. Link GitHub

PrakPBO_B_2511533002/PrakPBO_B_2511533002/src/Praktikum6/Tugas at Praktikum6 · rifqicraftplayz12-star/PrakPBO_B_2511533002